

GRChombo Coding Workflow Note

Development setup for the 5D Gregory-Laflamme black-string reproduction project

Version: 2026-05-11

Scope of this note

This note is about the coding and computing workflow, not the full physics research plan. It refers to the attached reproduction plan only to explain what the code eventually needs to support: a 5D black-string CCZ4 evolution, horizon diagnostics, and reproducible post-processing.

Contents

- 1. Executive summary
- 2. Recommended workflow architecture
- 3. What runs where: local PC, containers, Codex, GitHub, optional VM, HPC
- 4. Clean file-storage and version-control policy
- 5. Current reproduction goal and available GRChombo resources
- 6. What we still need to develop
- 7. Detailed Windows PC setup instructions
- 8. First smoke test after setup
- 9. Codex workflow and safe task design
- 10. Development milestones toward the black-string prototype
- 11. Practical commands and checklists
- 12. Sources and references

1. Executive summary

The cleanest daily development setup is a Windows x86_64 PC running Ubuntu through WSL2, with Docker Desktop using the WSL2 backend. GRChombo itself should be run inside the official GRChombo Docker image or a controlled native build later. The project repository should live inside the WSL Linux filesystem, not under C:\ or /mnt/c. VS Code connects to WSL, and Codex CLI runs from inside the repository.

The Mac can remain useful for notes, plotting, and lightweight post-processing, but Apple Silicon is not reliable for running the prebuilt amd64 GRChombo Docker executable. Codespaces has already proven useful as a temporary x86_64 smoke-test platform, but it is not ideal as a daily development machine because the session can stop when idle or disconnected. Eventually, production simulations should run on an HPC cluster; the local machine is for code development, small tests, and post-processing pipeline development.

Current status

We successfully ran the public BinaryBH very-cheap GRChombo example in a remote x86_64 environment, generated HDF5 output, installed Python plotting tools, patched the quick_plot script for the current yt API, and viewed PNG plots. This verifies the basic run-output-plot loop. The next goal is to reproduce that loop on the Windows PC and then use it as the stable development platform.

2. Recommended workflow architecture

The development stack should be layered. Each layer has a distinct responsibility, which keeps the setup reproducible and prevents large data or generated artifacts from polluting the source repository.

Windows PC, x86_64

- > WSL2 Ubuntu
 - > GitHub repository clone under ~/code/grchombo-smoke-test
 - > docs/, scripts/, src or dev/, analysis/, results summaries
 - > Docker Desktop WSL2 backend
 - > grchombo/grchombo container
 - > compile/run GRChombo examples and small tests
 - > Python virtual environment
 - > read HDF5, generate plots, run post-processing notebooks/scripts
 - > Codex CLI
 - > inspect code, create scripts/docs, make controlled edits, run tests

GitHub remote repository

- > persistent source of truth for code, docs, scripts, small plots, metadata

Optional persistent cloud VM

- > alternative x86_64 Linux development machine if Windows PC is unavailable

Future HPC cluster

- > large production runs only; outputs staged to scratch/project storage

This stack separates persistent source control from temporary execution. The repository stores the code and small reproducibility artifacts. Docker provides the GRChombo runtime. Raw numerical outputs live outside normal Git unless explicitly handled with Git LFS or external storage.

3. What runs where

Place	Used for	Should store	Should not store
Windows WSL2 repo	Daily development, scripts, docs, small tests, Codex work	Source code, docs, scripts, configs, small plots, metadata	Huge HDF5 dumps, virtualenvs, Docker images, build artifacts
GRChombo Docker container	Run or build GRChombo in a controlled Linux environment	Only temporary run files during execution	Long-term project state; copy outputs back to mounted repo/scratch before exiting
Python virtual environment	HDF5 reading, plotting, post-processing	Package environment files if needed; scripts in repo	The venv directory itself in Git
Codex CLI	Agentic code edits inside the repo	Diffs in the working tree; then Git commits	A separate project state outside Git
GitHub remote	Persistent version control and collaboration	Code, docs, scripts, small plots, metadata, maybe Git LFS pointers	Untracked raw simulation archives unless using LFS/external storage
Codespaces	Backup x86_64 smoke tests, quick cloud checks	Nothing unique; push changes back to GitHub	Only copy of important work
Cloud VM	Optional persistent remote Linux workstation	Clone of repo, scratch outputs	Irreplaceable source code not pushed to GitHub
HPC	Production-resolution simulations	Run directories, checkpoints, raw HDF5, logs	Unversioned code changes; use tagged commits instead

4. Clean file-storage and version-control policy

The main rule is: commit source, scripts, documentation, run metadata, and small human-readable outputs; keep raw numerical data in scratch space unless it is small or explicitly managed by Git LFS.

4.1 Suggested repository layout

```
grchombo-smoke-test/  
  README.md  
  .gitignore  
  docs/  
    workflow_note.pdf  
    grchombo_capability_map.md  
    section0_black_string_implementation_notes.md  
  scripts/  
    run_binarybh_smoke.sh  
    collect_binarybh_outputs.sh  
    plot_binarybh_outputs.sh  
    run_blackstring_toy.sh  
  dev/  
    BinaryBH_editable/          # optional scratch copy; consider not committing fully
```

```

    BlackStringToy/                # staged prototype example once stable
analysis/
    hdf5_inspection.py
    horizon_area.py
    rmin_tracking.py
results/
    BinaryBH_verycheap/
        metadata.txt
        plots/
        selected_logs/
runs/                               # gitignored raw run outputs
    BinaryBH_verycheap_hdf5/
    BlackStringToy_smoke/

```

4.2 Git policy

- Commit scripts, docs, parameter files, source patches, and small plots.
- Do not commit Python virtual environments, quick_plot/dev, object files, executables, HDF5 plot files, checkpoints, or large logs.
- Use one Git branch per meaningful development task, for example black-string-toy, source-map, or horizon-postprocessing.
- Make small commits with messages that say what was verified, for example: Add BinaryBH smoke-test script; verified HDF5 output on WSL2.
- For production runs, record the Git commit hash in metadata.txt and in the HPC run directory.

```

# Recommended .gitignore additions
.venv/
__pycache__/
*.pyc
quick_plot/dev/
*/quick_plot/dev/
*.o
*.d
*.ex
*.a
*.so
*.hdf5
*.3d.hdf5
hdf5/
checkpoints/
runs/

```

4.3 Metadata for every run

Every smoke test or physics run should include a small metadata file. This is more valuable than a screenshot because it lets us reproduce the environment later.

```

run_name: BinaryBH_verycheap
machine: Windows PC / WSL2 Ubuntu / Docker
architecture: x86_64
repo_commit: <git commit hash>
docker_image: grchombo/grchombo:<tag or digest if available>
run_command: ./Main_BinaryBH3d...ex params_very_cheap.txt
start_time_utc: ...
end_time_utc: ...
notes: produced HDF5; quick_plot patched set_window_size -> set_figure_size

```

5. Current reproduction goal and available GRChombo resources

The physics target comes from the attached reproduction plan: first reproduce the diagnostic backbone of the 5D Gregory-Laflamme black-string evolution, before attempting gravitational radiation. The relevant coding target in Section 0 is a 4+1-dimensional vacuum GR black string, compact z direction, CCZ4 evolution, $SO(3)$ modified cartoon reduction, 1+log lapse, integrated Gamma-driver shift, 6th-order finite differences, RK4 time stepping, 6th-order Kreiss-Oliger dissipation, GP black-string initial data, and horizon diagnostics.

5.1 What is already usable in public GRChombo

Resource	How it helps	Status
GRChombo framework	C++14 numerical relativity code using Chombo AMR, MPI/OpenMP, and modular examples.	Usable now
CCZ4/BSSN infrastructure	The public code includes CCZ4-related evolution infrastructure and moving-puncture-style gauge machinery.	Usable, needs source mapping in local clone
Examples: BinaryBH, KerrBH, ScalarField	Templates for parameter parsing, levels, AMR, output, and run structure.	Usable as scaffolding
Apparent horizon finder	Public source includes apparent-horizon finder components and string-like horizon geometry support.	Promising; must inspect interface
Docker image	Lets us reproduce a known GRChombo runtime without native dependency installation.	Already used successfully in x86_64 Codespaces
Python/yt/HDF5 plotting	Already verified for GRChombo output visualization after a minor quick_plot patch.	Usable now

Important limitation

We have not confirmed that the public GRChombo repository contains a ready-to-run 5D black-string/ $SO(3)$ modified-cartoon example matching Pau Figueras et al.'s setup. We are proceeding as a development project using public GRChombo building blocks, not as a simple parameter-file reproduction.

6. What we need to develop

The main missing work is the bridge between public GRChombo examples and the black-string system in the reproduction plan. This should be done incrementally, with a test after every small change.

Needed piece	Purpose	Development strategy	Validation
--------------	---------	----------------------	------------

BlackStringToy example	Separate example directory for staged development	Copy BinaryBH, rename minimally, preserve build	Build and run unchanged first
Source capability map	Know exactly which files implement CCZ4, gauge, AMR, output, AH finder	Use Codex/grep locally; write docs/grchombo_capability_map.md	Paths and responsibilities reviewed
GP black-string initial data	Set initial metric/extrinsic curvature/lapse/shift for uniform string	Implement analytic fields in a new initial-data class	t=0 fields smooth outside turduckening cutoff
GL perturbation in chi	Seed the unstable mode without changing asymptotic mass	Add epsilon $\sin(2 \pi n z/L) \exp[-(x/r_0 - r_0/x)^2]$	Perturbation localized near horizon and periodic in z
Compact z domain	Represent $z \sim z+L$	Set domain and boundary conditions carefully	Periodicity tests and no boundary artifacts in short run
4+1 CCZ4 + SO(3) modified cartoon	Actual 5D black-string dynamics reduced to 2+1 effective system	Hardest step; derive/code source terms and variables such as gamma_ww	Constraint damping, known stationary string, convergence
Horizon diagnostics	Compute A/A0, R_min, Fourier modes, generation times	Post-processing pipeline from horizon output	Reproduce clean diagnostics on small/medium tests
Run orchestration	Make simulations reproducible	Scripts for runs, collection, metadata, plotting	One-command smoke test

7. Detailed Windows PC setup instructions

These instructions assume the Windows PC is x86_64/AMD64. If the machine is ARM64, stop and choose a different platform.

7.1 Check architecture and virtualization

```
# PowerShell
$env:PROCESSOR_ARCHITECTURE
systeminfo
```

The architecture should be AMD64. In the systeminfo output, virtualization should be enabled in firmware. If virtualization is disabled, enable Intel VT-x/AMD-V/SVM in BIOS/UEFI.

7.2 Install WSL2 Ubuntu

```
# PowerShell as Administrator
wsl --install -d Ubuntu

# After restart if needed, open Ubuntu and update
```

```
sudo apt update
sudo apt upgrade -y

# PowerShell: verify WSL version
wsl -l -v
```

Ubuntu should show VERSION 2. If not, run `wsl --set-version Ubuntu 2` from PowerShell.

7.3 Install VS Code and connect to WSL

```
# In Ubuntu/WSL
mkdir -p ~/code
cd ~/code
code .
```

Install Visual Studio Code on Windows and the WSL extension. The lower-left of VS Code should say WSL: Ubuntu.

7.4 Install Docker Desktop with WSL integration

```
# In Ubuntu/WSL after Docker Desktop is installed and running
docker --version
docker run hello-world
```

In Docker Desktop settings, enable Use WSL 2 based engine and enable Ubuntu under Resources -> WSL Integration.

7.5 Clone the GitHub repository inside WSL

```
cd ~/code
git clone <YOUR_GITHUB_REPO_URL>
cd grchombo-smoke-test

git config --global user.name "Your Name"
git config --global user.email "your_email@example.com"
```

Keep the repo under `~/code` inside WSL. Avoid `/mnt/c/...` for this project because Linux filesystems are faster and avoid permission/symlink problems.

7.6 Install Python tools

```
cd ~/code/grchombo-smoke-test
python3 -m venv .venv
source .venv/bin/activate
pip install --upgrade pip
pip install yt h5py matplotlib numpy scipy
```

This venv is for post-processing and plotting, not for running GRChombo itself.

7.7 Install Codex CLI inside WSL

```
sudo apt update
sudo apt install -y nodejs npm
npm i -g @openai/codex

cd ~/code/grchombo-smoke-test
codex
```

Run Codex from the repository root so it can inspect and edit the project files. It does not create a separate persistent repo; Git is still the source of truth.

8. First smoke test after setup

The first goal on the Windows PC is to reproduce what worked in Codespaces: run the public BinaryBH very-cheap example, produce HDF5 output, generate a plot, and store only clean artifacts in the repository.

```
cd ~/code/grchombo-smoke-test
mkdir -p runs

docker pull grchombo/grchombo
docker run -v "$(pwd):/settings" -it grchombo/grchombo
```

Inside the GRChombo container:

```
cd BinaryBH
./Main_BinaryBH3d_ch.Linux.64.mpicxx.gfortran.DEBUG.OPT.MPI.OPENMPCC.ex params_very_cheap.txt

ls hdf5
cp -r hdf5 /settings/runs/BinaryBH_verycheap_hdf5
exit
```

Back in WSL:

```
cd ~/code/grchombo-smoke-test
ls runs/BinaryBH_verycheap_hdf5 | head
```

8.1 Plotting the smoke test

```
cd ~/code/grchombo-smoke-test/runs/BinaryBH_verycheap_hdf5
git clone https://github.com/ThomasHelfer/quick_plot.git
cd quick_plot
python3 -m venv dev
source dev/bin/activate
pip install --upgrade pip
pip install yt h5py matplotlib

# Patch old yt API usage if needed
sed -i 's/set_window_size/set_figure_size/g' parallel_pictures.py
python parallel_pictures.py

find . -path "./dev" -prune -o -name "*.png" -print | head
```

8.2 Save only pictures and metadata into the repo

```
cd ~/code/grchombo-smoke-test
mkdir -p results/BinaryBH_verycheap/plots
mkdir -p results/BinaryBH_verycheap/selected_logs

find runs/BinaryBH_verycheap_hdf5/quick_plot \
  -path "**/dev/**" -prune -o \
  -type f -name "*.png" \
  -exec cp {} results/BinaryBH_verycheap/plots/ \;

cat > results/BinaryBH_verycheap/metadata.txt <<'EOF'
run_name: BinaryBH_verycheap
platform: Windows WSL2 Ubuntu + Docker Desktop WSL2 backend
```



```

purpose: GRChombo smoke test, not physics production
raw_data_location: runs/BinaryBH_verycheap_hdf5/ (gitignored)
plots_location: results/BinaryBH_verycheap/plots/
EOF

git add results/BinaryBH_verycheap .gitignore
git commit -m "Add GRChombo BinaryBH smoke test plots and metadata"
git push

```

9. Codex workflow and safe task design

Codex should be used as a coding agent inside the repository. It can inspect files, write scripts, modify C++ code, and run tests. It should not be allowed to make uncontrolled physics claims or implement complicated numerical-relativity equations without review.

Rule

Give Codex small, testable tasks. Each task should end with a build, a run, a diff, or a document that we can review. Do not ask it to implement the full black-string code in one step.

9.1 Good first Codex prompts

- Inspect this repository and summarize the current smoke-test setup. Do not modify files yet.
- Create scripts/run_binarybh_smoke.sh, scripts/collect_binarybh_outputs.sh, and scripts/plot_binarybh_outputs.sh. They should reproduce the successful BinaryBH very-cheap run and save only plots and metadata in results/. Do not commit anything.
- Inspect the GRChombo source tree mounted in this workspace. Find the files responsible for CCZ4, gauge, parameter parsing, AMR tagging, output variables, apparent horizon finding, and initial data. Write docs/grchombo_capability_map.md. Do not modify C++ code.
- Create a branch black-string-toy. Copy BinaryBH into dev/BlackStringToy and perform only the minimal renaming needed so it builds as a separate example. Do not change the physics yet. Run make -j2 and report the result.

9.2 Unsafe Codex prompts

- *Implement the 5D black-string simulation completely.*
- *Derive and code the $SO(3)$ modified cartoon CCZ4 equations without review.*
- *Tune the simulation until it matches the paper.*
- *Delete unnecessary files or reorganize the whole repository automatically.*

10. Development milestones toward the black-string prototype

M0: Stable local environment: Windows WSL2 + Docker + VS Code + Codex + repo clone. BinaryBH smoke test runs and plots.

M1: Reproducible smoke-test scripts: One-command script can run BinaryBH very-cheap, collect metadata, and save plots.

M2: GRChombo source map: Document exact file paths for CCZ4, gauge, AMR, output, initial data, and AH finder in the local source tree.

M3: Editable build: Copy a public example into dev/BinaryBH_editable or dev/BlackStringToy and rebuild it successfully.

M4: BlackStringToy no-physics rename: New example builds and runs while still using inherited physics. This proves the scaffolding is correct.

M5: Uniform black-string initial-data prototype: Replace initial data only. Output $t=0$ fields and constraints. Do not attempt long evolution.

M6: Add GL perturbation and compact z: Implement epsilon, n , L , r_0 parameters and periodic z tests.

M7: 4+1/SO(3) cartoon evolution: Implement or adapt the actual reduced 5D evolution. This is the main numerical-relativity development step.

M8: Horizon and post-processing pipeline: Extract $h(t,z)$, γ_{xx} , γ_{xz} , γ_{zz} , γ_{ww} , $R(t,z)$, A/A_0 , $R_{\min}$, Fourier modes.

M9: L10 small/medium reproduction: Only after the above: attempt the first physically meaningful $L/r_0=10$ reproduction run.

11. Practical commands and checklists

11.1 Daily startup

```
# Open Ubuntu/WSL
cd ~/code/grchombo-smoke-test
git status
git pull
code .

# Optional: start Codex in repo root
codex
```

11.2 Run GRChombo container

```
cd ~/code/grchombo-smoke-test
docker run -v "$(pwd):/settings" -it grchombo/grchombo bash
```

11.3 Save clean progress

```
git status
git diff
# review changes carefully
git add docs scripts results/<small-results-only>
git commit -m "Describe the verified change"
git push
```

11.4 Smoke-test checklist

- `docker run hello-world` works in WSL.
- `grchombo/grchombo` container starts on Windows PC without Apple Silicon illegal-instruction problems.
- BinaryBH very-cheap executable runs.
- HDF5 files appear in the mounted output directory.

- Python/yt can load the HDF5 files.
- PNG plots are generated and can be opened in VS Code.
- Only plots and metadata are committed; raw HDF5 stays in runs/ or external storage.

12. Sources and references

The following sources were used for current tool/setup facts and for orienting the project to the attached research plan.

- User-provided file: G1 Black String Reproduction Plan.pdf. Used only as the physics target reference for Section 0 and the later diagnostics; this note does not reproduce the full plan.
- GRChombo GitHub repository: <https://github.com/GRTLCollaboration/GRChombo>
- GRChombo JOSS/arXiv overview: <https://arxiv.org/abs/2201.03458>
- Microsoft WSL install documentation: <https://learn.microsoft.com/en-us/windows/wsl/install>
- Docker Desktop WSL2 backend documentation: <https://docs.docker.com/desktop/features/wsl/>
- Docker Desktop WSL2 development workflow: <https://docs.docker.com/desktop/features/wsl/use-wsl/>
- VS Code WSL documentation: <https://code.visualstudio.com/docs/remote/wsl>
- OpenAI Codex CLI documentation: <https://developers.openai.com/codex/cli>